

How much data does PolyBench move? A reuse-distance study

Symbolic data-movement growth rates and coefficients for PolyBench, under single-shot and infinitely-repeating execution, with empirical confirmation

2026-07-08

This report measures the **data-movement cost** of the PolyBench kernels and explains what drives it. We use the AutoLALA dmd analyzer, which reads a loop nest and produces *symbolic* formulas — cost as a function of the problem size, not a single number for one input. From those formulas we extract a simple, testable law for how each kernel’s data movement grows, rank the kernels by both growth rate and constant factor, and check the predictions against real cache-miss counts and wall-clock time.

Everything here is reproducible; see **Reproduce** at the end.

1. Three quantities, in plain words

When a program runs, it touches memory in some order. To reason about locality we track, for each piece of data, *how long it sits unused between two touches*. There are two natural ways to measure “how long”, and one summary number built on top of them.

Reuse interval (RI). When you touch a value and later touch it again, the reuse interval is *how many memory accesses happened in between*, counted in accesses. If you read $A[0]$, then read nine other things, then read $A[0]$ again, the reuse interval of that second $A[0]$ is 10.

Reuse distance (RD). The reuse interval counts *every* access in between; the reuse distance counts only the *distinct cache lines* touched in between. That is what actually decides whether the value is still in cache when you return to it. If those nine intervening accesses only ever hit two different cache lines, the reuse distance is 2–3 lines even though the reuse interval is 10. RD is the working-set size between two touches, in cache lines.

A cache line here is **8 doubles** (a 64-byte line holds eight 8-byte f64 values). All PolyBench arrays are double precision, so the analyzer’s block size is 8 elements. (Block size is in *elements*, not bytes.)

Data-movement distance (DMD). This is the single summary number. For every reuse in the program it adds the **square root of that reuse’s distance**:

$$\text{DMD} = \sum_{\text{reuses}} \sqrt{\text{reuse distance}}.$$

Why the square root, and why the sum? The model treats a reuse at distance d as costing $\sqrt{d}$ to service — a memory-hierarchy cost model in which data further away (a larger working

set to traverse before you get back) is more expensive, but sub-linearly so. Summing over all reuses gives one number whose *growth rate* captures how a kernel’s data traffic scales with problem size. A kernel whose reuses stay at small distance has low DMD; one that keeps reaching far back into a large working set has high DMD. We report DMD **as a formula in the problem size**, then read off its growth rate.

2. Two models of the trace: single-shot vs. infinitely repeating

A kernel’s reuse pattern depends on an assumption people rarely state: *does the kernel run once, or over and over?*

Single-shot. The kernel runs exactly once. The very last time a value is touched it is never touched again, so that touch is a **cold miss** (compulsory — the data had to be brought in at least once). This is the right model for a one-off computation.

Infinitely repeating. The kernel runs many times back-to-back — think of a linear-solver step, a stencil sweep, or an inner loop re-run every outer iteration. Now the “last” touch of a value in one pass is followed by the “first” touch of the same value in the next pass, so that value is reused — at a large distance (you sweep the whole working set in between), but reused nonetheless. This is the right model for a kernel that lives inside an outer iteration.

We implement infinite repeat the simple way: wrap the kernel in an outer loop that runs **twice**, and keep only the reuse intervals whose *second* touch lands in the second pass. That pass has a full period of history behind it, so its reuses — including the ones that wrap around the period boundary — are the steady-state ones. Two passes suffice whenever every value live in one pass is touched again in the next (true for all these kernels).

Here is the difference on a two-line example — read $A[i][j]$ and $B[j]$ inside an i, j nest. $B[j]$ is reused every i (short distance); $A[i][j]$ is touched exactly once per pass.

model	reuse distances the analyzer reports
single-shot	$2 \cdot M, 4, 2$
infinite-repeat	$2 \cdot M, 4, 2, N \cdot M + M, N \cdot M + 2, N \cdot M + 1$

Single-shot sees only B ’s short reuse and calls every $A[i][j]$ a cold miss. Infinite-repeat additionally sees that each $A[i][j]$ is re-touched next pass — at a reuse distance of about $N \cdot M$, the size of the whole array. The access *total* is identical either way ($2 \cdot N \cdot M$); infinite-repeat does not invent work, it reclassifies boundary cold misses as far-away reuses. **This is the extra information infinite-repeat gives: reuse that only exists across invocations.** We report both models throughout.

3. The growth-rate law

Write the problem size as N . Two growth rates matter:

- **access growth** a — the number of memory accesses grows like N^a (a triple loop over N does N^3 accesses, so $a = 3$).
- **reuse-distance growth** ρ — the *largest* reuse distances grow like N^ρ .

Because DMD sums $\sqrt{\text{distance}}$ over roughly N^a reuses, and the dominant reuses sit at distance $\sim N^\rho$ each contributing $\sqrt{N^\rho} = N^{\rho/2}$, the DMD grows like

$$\text{DMD} \sim \text{coeff} \cdot N^d, \quad d = a + \frac{1}{2}\rho.$$

In words: **the DMD growth exponent is the access-growth exponent plus half the reuse-distance-growth exponent.** We compute d directly and stably from the reuse-distance distribution (summing the square-root terms), which avoids the numerical cancellation you get from evaluating the fully assembled formula.

We call $d - a = \frac{1}{2}\rho$ the **headroom**. It is the most useful single number here, so plainly:

Headroom is how much *faster* a kernel’s data movement grows than its raw arithmetic. Headroom 0 means data movement grows in lockstep with work — the kernel is already as local as it can be. Large headroom means the kernel moves far more data than it computes, and a locality transformation (tiling, loop interchange) has room to recover roughly a factor of N^{headroom} .

Across the symbolic kernels ρ takes only a few clean values, so headroom clusters at a few levels — the backbone of the taxonomy in §6.

4. Same growth rate, different constant: the coefficient

Growth rate alone cannot separate two kernels that scale the same way. Two kernels can both be $\Theta(N^4)$ yet move meaningfully different amounts of data, because the **leading coefficient** in $\text{DMD} \sim \text{coeff} \cdot N^d$ differs. The coefficient is physically meaningful — it counts how many square-root-of-distance units of traffic the kernel issues per N^d — so we report it and use it to rank kernels *within* a growth class.

We extract the coefficient from the same square-root construction used for the exponent: for each dominant reuse-distance term, take the leading coefficient of its multiplicity times the square root of the leading coefficient of its distance, and sum the terms sharing the top growth order. The result is a single number coeff with $\text{DMD} \approx \text{coeff} \cdot N^d$ at large N .

The ranking is intuitive where we can check it by hand. `gemm`, `2mm`, and `3mm` all grow as N^4 (headroom 1.0), but their coefficients are 0.044, 0.088, and 0.132 — almost exactly 1 : 2 : 3, because `2mm` is two chained matmuls and `3mm` is three, each contributing one matmul’s worth of far-reaching reuse. The exponent says “these are all N^4 kernels”; the coefficient says “`3mm` moves three times the data of `gemm`.” Likewise in the flat headroom-0 band, `syr2k` (coeff 3.79) moves about $4\times$ the data of `cholesky` (0.92) at the same N^3 growth. We therefore rank kernels first by headroom, then by coefficient.

5. Results — how every kernel scales

Below are all 27 symbolic kernels the analyzer could handle, grouped by headroom. For each: the access growth a , the DMD growth d , the headroom $d - a$, and the leading coefficient (so $\text{DMD} \approx \text{coeff} \cdot N^d$). The last two columns repeat d and the coefficient under the infinite-repeat model; “—” means the doubled trace exceeded the analyzer’s counting budget (see §7). A **bold** inf-repeat order marks a kernel whose growth rate *rises* under repetition.

Headroom 1.0 — data movement grows a full factor of N faster than the work.

kernel	a	d	headroom	coeff	d (inf)	coeff (inf)
doitgen	4	5.0	1.0	0.044	5.0	0.044
jacobi-2d	3	4.0	1.0	0.468	—	—
3mm	3	4.0	1.0	0.132	4.0	0.132
2mm	3	4.0	1.0	0.088	4.0	0.088
seidel-2d	3	4.0	1.0	0.083	—	—
gemm	3	4.0	1.0	0.044	4.0	0.044
floyd-warshall	3	4.0	1.0	0.044	4.0	0.044

Headroom 0.5 — data movement grows a factor of $\sqrt{N}$ faster than the work.

kernel	a	d	headroom	coeff	d (inf)	coeff (inf)
gramschmidt	3	3.5	0.5	2.685	3.5	2.098
imperfect	3	3.5	0.5	2.568	3.5	2.656
covariance	3	3.5	0.5	0.706	3.5	0.706
correlation	3	3.5	0.5	0.706	—	—
fdtd	3	3.5	0.5	0.242	—	—
gemver	2	2.5	0.5	1.216	2.5	1.216
mvt	2	2.5	0.5	1.063	2.5	1.063
jacobi-1d	2	2.5	0.5	0.249	2.5	0.250
bicg	2	2.5	0.5	0.153	3.0	0.044
gesummv	2	2.5	0.5	0.076	3.0	0.125
atax	2	2.5	0.5	0.062	3.0	0.044

Headroom 0.0 — as read by the simple order estimate. These are all triangular kernels, and this is where the simple “order at large N ” is least reliable. **§10 shows most of these readings are too low:** the exact trace puts syr_k, syr_{2k}, symm, cholesky, lu at positive, still-climbing headroom, and trmm at +1 once bin masses are summed exactly. Read this table as “what the smooth model reports,” not “the truth,” for the triangular kernels — the † marks the ones §10 corrects.

kernel	a	d	headroom	coeff	d (inf)	coeff (inf)
syr2k†	3	3.0	0.0	3.789	3.0	3.827
symm†	3	3.0	0.0	2.899	3.0	2.943
syrk†	3	3.0	0.0	2.158	3.0	2.191
lu_decomp†	3	3.0	0.0	1.899	3.0	1.881
lu†	3	3.0	0.0	1.378	—	—
trmm†	3	3.0	0.0	1.366	3.0	1.351
cholesky†	3	3.0	0.0	0.923	3.0	0.940
trisolve	2	2.0	0.0	2.871	3.0	0.031

Two things stand out for the kernels whose order *has* anchored (headroom 1.0 and 0.5, and trisolve here). First, **every such headroom value is 0.0, 0.5, or 1.0** — nothing in between; the reuse-distance growth ρ is quantized to $\{0, 1, 2\}$, with no kernel whose worst reuse distance grows like, say, $N^{1.3}$. Second, within a band the **coefficients spread over 1-2 orders of magnitude**, which is exactly the ranking information the growth rate alone throws away (§4). The catch — that the quantization looks *cleaner* than it is because the triangular kernels were mis-binned into headroom 0 — is the subject of §10.

6. Three optimization classes

Because headroom takes only three values, the kernels fall into three groups, and the group tells you *which* locality transformation can help and *how much*.

Class A — headroom 1.0 (reuse distance grows like N^2). The dense linear-algebra and 2-D stencil kernels: gemm, 2mm, 3mm, doitgen, floyd-warshall, jacobi-2d, seidel-2d. Here the working set reached between reuses is a whole matrix, so the reuse distance grows as N^2 and the data movement outpaces the arithmetic by a full factor of N . The right transformation is **tiling**: block the loops so the reused sub-matrix fits in cache, which caps the reuse distance at the (constant) tile footprint and collapses the headroom to zero. This is the largest asymptotic win available — and the one the empirical section confirms most sharply.

Class B — headroom 0.5 (reuse distance grows like N). Matrix-vector products and mixed 1-D patterns: mvt, atax, bicg, gemver, gesummv, jacobi-1d, covariance, correlation, fdtd, gramschmidt, imperfect. The culprit is typically one array streamed “the wrong way” — read down a column while it is stored by rows, or re-streamed on each pass — so its reuse distance grows as N . The fix is **loop interchange or fusion** to bring that access’s reuse close, recovering about $\sqrt{N}$.

Class C — headroom 0.0 (reuse distance bounded), as the *scale model* reports it. The kernels the model puts here are the triangular ones: syr, syr2k, cholesky, lu, lu_decomp, trmm, trisolve, symm. Taken at face value the reading is “reuse is already local, no asymptotic slack, don’t bother tiling.” **This label is mostly wrong, and §10 shows why.** On a triangular iteration space the leading DMD term is hard to anchor and the scale approximation under-resolves the largest reuse distances, so it reports headroom 0 for kernels that actually reuse data at whole-matrix distance. The exact trace puts syr, syr2k, symm, cholesky, lu with *positive*, still-climbing headroom (and trmm at +1 once bin masses are summed correctly) — they do reward tiling, just past a larger cache threshold. The only kernel that stays genuinely near the bottom is trisolve (reuse distance $\sim N$, and even that is promoted by repetition, §7). **Treat a headroom-0 reading on a triangular kernel as provisional and check its curvature (§10) before concluding there is nothing to optimize.**

The point of the taxonomy is predictive: given a new affine kernel, its headroom tells you up front whether a locality transformation is worth attempting at all, and if so which one and roughly what payoff to expect — *provided the order has anchored*, which §10 shows is not automatic on triangular nests.

7. What infinite-repeat reveals

Comparing the two models across the whole suite gives a clean answer to “does infinite-repeat tell us anything the single-shot model doesn’t?” — **yes, for a specific and recognizable set of kernels.** Four kernels raise their DMD growth rate under repetition:

kernel	single-shot	infinite-repeat
atax	$N^{2.5}$	$N^{3.0}$
bicg	$N^{2.5}$	$N^{3.0}$
gesummv	$N^{2.5}$	$N^{3.0}$
trisolve	$N^{2.0}$	$N^{3.0}$

These are exactly the kernels that **read their matrix once per invocation**. In a single pass, each matrix element is touched once and then never again, so the single-shot model files it

as a cold miss and it contributes nothing to the reuse-driven part of DMD. But if the kernel repeats — `atax` inside a Krylov solver, `trisolve` inside a Newton iteration — the matrix is read again on the next pass, a genuine reuse at distance $\approx N^2$ (the matrix footprint). That adds a term of size (number of matrix elements $\sim N^2$) $\times \sqrt{N^2} = N^3$, lifting the DMD order. Infinite-repeat surfaces this cross-invocation reuse; single-shot cannot see it.

The contrast within Class B is telling. `mvt` and `gemver` are also matrix-vector kernels, yet they do **not** jump — because they already touch the matrix more than once per pass (`mvt` multiplies by A and $A^\top$; `gemver` applies rank-1 updates), so their matrix reuse is captured within a single pass and repetition adds nothing new to the leading term. So the jump is a real signal: it distinguishes “the matrix is streamed once and only reused across invocations” from “the matrix is reused within each invocation.”

Practically: if you are optimizing one of the jumping kernels *inside an outer iteration*, the analysis says the matrix reuse across iterations is worth capturing (keep the matrix resident if it fits), whereas the single-shot view would have told you the matrix traffic is unavoidable cold-miss volume.

Where infinite-repeat did not complete (`jacobi-2d`, `seidel-2d`, `correlation`, `fdtd`, `lu` — the “—” rows in §5), the doubled trace pushed the Barvinok counter past its operation budget. For every kernel where it did complete, the headroom stayed the same or rose — never fell — consistent with the model only ever *adding* boundary reuses, never removing intra-pass ones.

8. Empirical confirmation

The growth-rate law is a claim about reuse distances, and reuse distance is what decides cache behavior. So we check the predictions on real hardware with the actual C kernels (not the model): **cache-miss scaling** with `cachegrind`, and **wall-clock time per access** over a size sweep. We take one kernel from each class. The prediction in each case is about the *last-level* miss rate, because that is what a growing reuse distance eventually blows.

Class A — `matmul` (predicted headroom 1: reuse distance $\sim N^2$). As N grows, the reused operand (a full $N \times N$ matrix) eventually overflows last-level cache. The last-level miss rate should stay tiny while it fits, then jump once it doesn’t — and tiling, which caps the reuse distance at the tile size, should prevent the jump.

last-level miss rate	N=128	N=256	N=384	N=512
naive	0.18%	0.08%	0.07%	3.55%
tiled	0.12%	0.05%	0.04%	0.16%

The naive kernel’s miss rate is flat and small up to $N=384$, then **jumps about $50\times$** at $N=512$ — exactly when the working set crosses the simulated 2 MB cache. Tiling holds it at 0.16%, **$22\times$ lower** than naive at $N=512$ and still flat. Wall-clock time per access rises with the miss rate — 1.67, 2.01, 2.15 ns at $N = 256, 512, 1024$ — then saturates at memory latency. This is the headroom-1 signature: an unbounded reuse distance that tiling removes.

Class B — `mvt` (predicted headroom 0.5: reuse distance $\sim N$). One array is streamed transposed; its reuse distance grows as N , so the miss rate should climb with N until that access no longer fits, and loop interchange should flatten it.

last-level miss rate	N=512	N=1024	N=2048
naive	9.9%	31.0%	31.2%
interchanged	7.4%	7.5%	7.5%

The naive miss rate **grows** from 9.9% to 31% as the transposed access outgrows cache; loop interchange fixes the access order and pins it at 7.5% across all sizes — about **4× lower** at $N=2048$. Headroom-½ signature: a growing reuse distance from one mis-ordered access, capped by interchange.

Class C — syrk (scale model predicts headroom 0). The scale model says reuse is bounded, so the miss rate should be flat. Over the sweep we ran, it is:

last-level miss rate	N=128	N=256	N=384	N=512
naive	0.31%	0.13%	0.12%	0.09%

But this “confirmation” is a size-range artifact, not a headroom-0 signature — see §10. The sweep stopped at $N = 512$. `syrk` actually reuses data at whole-matrix distance just like `gemm`; its cache cliff is simply at a slightly larger size, because its triangular structure shrinks the reuse distance by a constant factor. Extending the exact trace one octave further, `syrk`’s last-level miss rate jumps from 0.04% at $N = 512$ to 1.86% at $N = 640$ — the same two-order-of-magnitude cliff `gemm` shows at 512. So `syrk` is *not* a headroom-0 kernel; §8’s flat table caught it just before its crossing. The honest Class-C example is instead `trisolve`, whose reuse distance genuinely grows only like N .

The Class A and B predictions (a cliff that tiling removes; a climbing rate that interchange flattens) are observed cleanly on real cache hardware. The Class C prediction is the cautionary one: a flat miss curve confirms headroom 0 only if the sweep actually reaches the kernel’s cache threshold — which §10 shows requires checking the reuse distance, not just eyeballing the range you happened to run.

A note on ground truth. `Cachegrind` is used only as an *independent check of the reuse-distance prediction* — does the miss rate move the way the model says? — not as a performance oracle. Wall-clock time is the performance measure. (PMU counters via `perf` were unavailable on this host, which runs with `perf_event_paranoid=4`; we did not weaken that host setting, and `cachegrind` plus timing were sufficient.)

9. Finite ranges: the local exponent, and what doubling N actually costs

Everything up to here is asymptotic. In practice you have a size *range* and you ask: “if I double N from here, what does it cost?” The order d answers only the limit — it says DMD scales by 2^d per doubling. Over a real octave $[N, 2N]$ the honest quantity is the **local exponent**, the log-log slope measured right at your size,

$$p(N) = \frac{\log(\text{DMD}(2N)/\text{DMD}(N))}{\log 2},$$

computed exactly from the symbolic formula (`local_analysis.py`, with $\text{DMD}(N)$ evaluated as $\sum \text{multiplicity} \cdot \sqrt{\text{distance}}$ over the reuse-distance bins — no cancellation). Here is `gemm` (Class A, order $d = 4$):

N	$\text{DMD}(2N)/\text{DMD}(N)$	$\text{local } p(N)$	naive 2^d	access slope	local headroom
64	11.14	3.48	16.0	2.99	0.48
256	13.53	3.76	16.0	3.00	0.76
512	14.47	3.85	16.0	3.00	0.86
1024	15.09	3.92	16.0	3.00	0.92
4096	15.70	3.97	16.0	3.00	0.97

Two things to read off. First, the naive 2^d **overestimates** the real cost of doubling at practical sizes: at $N=64$, doubling multiplies DMD by $11.1\times$, not the asymptotic $16\times$; the local exponent 3.48 is the honest growth rate there, and it only creeps toward 4.0. Second, the **local headroom** (last column) starts at 0.48 and climbs to 0.97 — so at small N gemm’s per-access data movement grows far less than the asymptotic headroom of 1.0 promises. You are *pre-asymptotic*: the tiling payoff is latent, and how latent depends on where your N sits.

Contrast what the *scale model* shows for syrk:

N	$\text{DMD}(2N)/\text{DMD}(N)$	$\text{local } p(N)$	naive 2^d	local headroom
64	7.97	2.99	8.0	0.011
256	7.99	3.00	8.0	0.003
4096	8.00	3.00	8.0	0.000

In the scale model syrk sits *at* exponent 3.00 with zero local headroom, and one would read this as “no growing reuse distance, no latent payoff.” §10 shows **this is the scale approximation under-resolving syrk’s reuse: the exact trace has its local exponent at 3.5-3.6 and climbing**. The lesson of this table is therefore double-edged — the local exponent is exact *for the model you feed it*, but if that model has compressed the reuse distances (as the scale approximation does on triangular nests), an exact local exponent of a wrong curve is still wrong. Curvature is the guard: the exact syrk curve has $q > 0$ throughout, flagging the un-anchored term; the scale curve reports a spurious $q = 0$.

The cliff the smooth exponent hides. DMD is a smooth polynomial, so its local exponent drifts continuously. The real *miss* curve does not — it has a cliff at the size where the dominant reuse distance fills the cache. gemm’s dominant reuse-distance term is $0.125 N^2$ cache lines, i.e. $0.125 N^2 \times 64 \text{ B} = 8N^2$ bytes, which fills a 2 MB last-level cache at

$$N^* = \sqrt{2 \text{ MB} / 8 \text{ B}} = 512.$$

The cachegrind sweep in §8 put gemm’s last-level miss-rate jump at **exactly** $N=512$. The model’s reuse-distance *coefficient* predicted the measured cliff location, not merely its existence. syrk has the *same* kind of $\sim N^2$ dominant distance — the scale model just could not see it — so it has a real cliff too, near $N \approx 560$ (§10); the smooth headroom-0 curve above hides it entirely. This is why the threshold check must be run against the *exact* reuse distance, not the scale model’s compressed one, whenever curvature says the order has not anchored.

So the complete finite-range recipe is: use the **local exponent** $p(N)$ (exact, from the formula) for the smooth cost, and separately test whether your octave $[N, 2N]$ **straddles** $N^* = \sqrt{C/c}$ for the cliff. The derivative refines the exponent; the threshold catches the discontinuity the exponent cannot see.

10. Re-examining the headroom-0 class: anchoring, curvature, and the exact trace

The three-class taxonomy has a soft spot exactly where §9 was most confident. The kernels the model calls “headroom 0” are almost all **triangular** — `syrk`, `syr2k`, `symm`, `cholesky`, `lu`, `trmm`, `trisolve`. On a triangular iteration space two things conspire to make the leading DMD term hard to read, and both push the *reported* headroom below the *true* one.

Why the order is hard to anchor. A triangular loop nest produces reuse-distance bin populations that are quasi-polynomials like $\frac{1}{6}N^3 - \frac{1}{2}N^2 + \dots$ — a large leading term and large, opposite-sign lower terms. Fitting a single “order at large N ” to such a curve converges slowly, and worse, a genuinely faster-growing DMD term can hide underneath with a *small coefficient*, so it does not overtake until N is in the thousands. Reading one exponent off the formula then reports the bulk term and misses the faster one entirely. This is precisely the situation the user anticipated: “those loops are triangular and it is hard to anchor the leading term.”

We characterize it three ways instead of one (`anchor_analysis.py`).

1. An anchored term spectrum, exact. We first made the analyzer emit, per reuse-distance bin, an exact **mass** — the parameter-only bin population — rather than a per-iteration-point count that has to be summed by hand (a fix that also corrected the warm/compulsory split; see §12). With exact masses, we anchor each term with no fitting at all: on the residue class $N = N_0 + 8k$ (step 8 absorbs the cache-line periodicity) a mass is an exact polynomial, so finite differences over rational arithmetic give its **exact degree and leading coefficient**. Summing $\text{mass} \cdot \sqrt{\text{distance}}$ term by term yields the DMD *spectrum*. The hidden term becomes visible — for `syrk`,

$$\text{DMD}_{\text{scale}}(N) \approx 2.77 N^3 + 0.022 N^{3.5} + \dots,$$

an $N^{3.5}$ term (headroom $+\frac{1}{2}$) with a coefficient $120\times$ smaller than the bulk, so it only overtakes near $N \approx 16,000$. The naive “order at large N ” reads 3 and calls the headroom 0; the spectrum shows the headroom is positive but *latent*.

2. Curvature as the anchoring diagnostic. A pure power law is a straight line in log-log axes, so the discrete second derivative

$$q(N) = p(2N) - p(N), \quad p(N) = \log_2 \frac{\text{DMD}(2N)}{\text{DMD}(N)},$$

is zero once the leading term has anchored, and *positive* while a faster term is still mixing in. Curvature is the model-independent answer to “should I trust this exponent?” — $q \approx 0$ means yes; $q > 0$ and rising means the reported order is a lower bound. Across the triangular kernels q is persistently positive (+0.05 to +0.14); across the kernels whose order *has* anchored it sits at zero. This is exactly the “second-order/curvature” quantity worth having, and in DMD it is not a curiosity — it is the flag that separates a trustworthy order from a premature one.

3. The exact trace as arbiter. To settle what the scale approximation itself gets wrong, we generate a C simulator directly from each kernel’s DSL and compute **exact** reuse distances over 64-byte lines with a Fenwick-tree stack-distance pass (`exact/gen_sim.py`; validated to the digit against brute force). No approximation, no formula. The result is decisive:

kernel	scale headroom	exact $p(N)$, small→large N	curvature q	exact/scale DMD ratio	max reuse dist.: exact vs scale
syrk	0.0	3.45 → 3.63 ↑	+0.07	1.5 → 5.2	2079 vs 20 lines
syr2k	0.0	3.48 → 3.55 ↑	+0.07	1.8 → 5.2	huge gap
symm	0.0	3.48 → 3.66 ↑	+0.07	2.1 → 7.5	huge gap
cholesky	0.0	3.34 → 3.66 ↑	+0.13	0.9 → 2.9	1087 vs 24 lines
lu	0.0	3.48 → 3.61 ↑	+0.05	1.9 → 4.6	large gap
trmm	+1.0	3.53 → 3.59 ↑	+0.02	0.83 (flat)	matches
trisolve	0.0	2.09 → 2.25 ↑	+0.04	~flat	small
gemm (ref)	+1.0	3.34 → 3.73 ↑	+0.15	~flat	matches
mvt (ref)	+1.0	2.43 → 2.79 ↑	+0.08	~flat	matches

Read the table across. Wherever the exact and scale models **agree** (trmm, gemm, mvt), the exact/scale DMD ratio is flat and the two models resolve the same maximum reuse distance — the scale headroom is trustworthy. Wherever the scale model reports **headroom 0** for a triangular kernel, its ratio to the exact DMD *grows* with N (up to $7.5\times$), and it resolves reuse distances only up to ~ 20 lines when the true ones reach $N^2/8$ lines — hundreds to thousands. The scale approximation is **blind to these kernels’ whole-matrix reuse**, and its headroom-0 label is an artifact.

Two corrections to the taxonomy follow. First, the exact-mass fix alone moves **trmm out of Class C** — with masses summed correctly it plainly carries an N^4 term (headroom +1), which the scale model does capture. Second, the exact trace moves **syrk, syr2k, symm, cholesky, lu** out of “no locality slack”: their DMD grows faster than their access count (exact exponent already 3.5–3.7 and climbing, versus the scale model’s flat 3.0), because they reuse data at whole-matrix distance just like gemm. trisolve is the one kernel that stays genuinely near the bottom — its reuse distance grows only like N , and even its mild drift is the vector reuse that infinite-repeat (§7) promotes.

This is not academic — the cache cliff is real, §8 just stopped short. §8 read syrk’s flat measured miss rate as confirmation of headroom 0. But the same exact trace, asked for capacity misses against a 2 MB last-level cache, shows the cliff is simply at a larger size:

last-level miss rate (2 MB, exact)	N=384	N=448	N=512	N=640	N=768
gemm	0.02%	0.02%	3.13%	3.13%	3.13%
syrk	0.02%	0.02%	0.04%	1.86%	2.52%

syrk has the *same* cliff as gemm — its miss rate jumps two orders of magnitude once its whole-matrix reuse outgrows the cache — only it crosses near $N \approx 560$ instead of 512, because

the triangular structure makes its reuse distance a constant factor smaller. §8’s cachegrind sweep stopped at $N = 512$, exactly in the gap where `gemm` has crossed and `syrk` has not yet, which is why `syrk` looked flat. The “nothing to optimize” reading was the miss-rate twin of the un-anchored exponent: **both were finite-size artifacts, and curvature plus the exact trace expose both.**

The practical upshot: for a triangular BLAS-3 kernel, do not trust a headroom-0 reading from the smooth model. Check the curvature; if it is positive and rising, the kernel has latent locality slack and *will* reward tiling once N is large enough to cross its (later, but real) cache threshold.

11. What we could and could not analyze

Of the 53 PolyBench programs (31 symbolic-size, 22 fixed-size), the analyzer handled **48 under the single-shot model**, and **43 of those also under infinite-repeat**. That single-shot figure is up from 41 before this work: the reduction-loop fix in §12 recovered the six accumulator kernels (convolution, `symm`, `gramschmidt` in both size families) that the extractor had previously refused. The growth-rate table in §5 covers the 27 symbolic kernels that yield a size-dependent formula (the fixed-size kernels give one number, not a growth rate, and are used only as numeric cross-checks).

The five programs we could not analyze fall into three honest buckets:

- **Not representable** — `adi`, `deriche`, `durbin`. These use genuinely non-affine array subscripts (an index computed from loaded data) or MLIR syntax the current toolchain no longer parses. They are outside the affine model, and we let them error rather than silently drop accesses. (None were in the legacy AutoLALA reference set either.)
- **Over the counting budget** — `heat-3d`. A deep, heavily-parametric 3-D stencil whose reuse relation exceeds the Barvinok operation budget even single-shot.
- **Excluded** — `fdtd-apml`, a known pathological case for the counter (it times out); dropped by request.

Infinite-repeat additionally could not complete on the five heaviest kernels that *do* analyze single-shot (`jacobi-2d`, `seidel-2d`, `correlation`, `fdtd`, `lu`): doubling the trace doubles the outer schedule dimension and pushes the polytope past the counting budget. One further kernel, `convolution`, analyzes but its symbolic bounds (N - K in both loop dimensions) leave the access-count formula without a clean single growth order, so it is omitted from the ranked table.

None of these gaps affect the central findings, which rest on the 27 symbolic kernels that do produce clean formulas.

12. Implementation notes

Two toolchain changes made this study possible; both live in this branch.

Loading kernels with reductions. The MLIR extractor had been rejecting any loop that carried a value across iterations (an `iter_args` reduction), which blocked every kernel with an inner accumulator (`convolution`, `symm`, `gramschmidt`). The DMD model only tracks memory movement, so a value carried in a register is irrelevant — the legacy AutoLALA extractor simply ignored such carried values. We restored that: an `affine.for` with `iter_args` is treated as an ordinary loop, its carried scalar and `affine.yield` ignored, its inner loads/stores flowing through unchanged. Scratch (de)allocations (`memref.alloc/alloca/dealloc`) are likewise

ignored. This recovered the six reduction kernels. Kernels with genuinely non-affine subscripts (*adi*, *durbin*, *deriche*) remain out of scope — not representable — and we let them error rather than silently drop accesses.

The infinite-repeat model. We added `--infinite-repeat` to `dmd-cli`. It wraps the program in a two-iteration outer loop and keeps only the reuse intervals whose consuming access lands in the second iteration, as in §2. This is simpler than the original AutoLALA scheme (no symbolic repeat count, no normalization): two concrete passes suffice and the second-pass filter yields the steady-state distribution directly. A unit test pins the behavior to the §2 example — the wraparound reuse must appear at footprint distance and the access total must stay at one period.

Exact reuse-distance bin mass. Each reuse-distance bin previously reported only per-region *counts*, and when a bin’s value depended on loop iterators those iterators were lifted into parameter space, so the count became per-point and a naive read under-counted the bin population. We added an exact, parameter-only **mass** per bin — the cardinality of the un-projected piece domain, from `isl` — which fixed the warm/compulsory split and is what the §10 anchoring relies on.

Exact-trace simulator. `exact/gen_sim.py` parses each kernel’s DSL, emits its loop nest as C, and computes exact stack distances with a Fenwick tree over 64-byte lines, reporting DMD, the reuse-distance histogram, the maximum reuse distance, and capacity misses against 2 MB / 8 MB caches. It is the approximation-free ground truth used in §10; validated to the digit against a brute-force reference on small sizes.

Block size. The analyzer’s block size is in elements. A 64-byte cache line is 8 doubles, so we pass `--block-size 8` (16 would model single precision). This matches the line `cachegrind` simulates.

Reproduce

```
python3 run_analysis.py both --resume      # analyze all kernels, both models -> results/
python3 analyze_math.py                   # growth rates + coefficients -> order_table.json
python3 local_analysis.py                 # local exponent / doubling cost / cache threshold N*
python3 anchor_analysis.py                # exact-anchored spectrum + curvature vs exact trace ($)
cd exact && python3 gen_sim.py              # exact stack-distance trace -> exact/<kernel>.json
cd confirm && python3 sweep.py              # cachegrind miss-scaling -> cg.json
pandoc REPORT.md -o REPORT.pdf --pdf-engine=xelatex \
    -V mainfont="DejaVu Serif" -V monofont="DejaVu Sans Mono"
```